Comprehensive Software Requirements Specification and Agentic Implementation Protocol for Video-Based Digital Loan Origination

The traditional landscape of digital financial onboarding is perpetually hindered by intrinsic friction points that compromise both the user experience and the institutional risk posture. Historically, digital lending journeys have relied upon static, form-based application architectures that necessitate manual data entry, asynchronous document uploads, and disconnected verification workflows. These legacy systems invariably experience high application abandonment rates, as prospective borrowers become fatigued by repetitive data requests and protracted approval latencies.[1] Furthermore, purely form-based onboarding mechanisms expose financial institutions to elevated vectors of identity fraud and misrepresentation, as they inherently lack the capacity to capture real-time behavioral telemetry, validate the physical presence of the applicant, or actively authenticate the contextual intent behind the loan application.[1] The persistent reliance on manual Know Your Customer (KYC) verification processes introduces severe operational bottlenecks, scaling limitations, and extensive administrative overhead, precluding the possibility of instant, automated decision-making.[1]

To resolve these systemic inefficiencies, modern financial institutions require highly secure, fully compliant, and artificially intelligent origination architectures. The proposed solution, conceptualized as the Poonawalla Fincorp Loan Wizard, represents a fundamental paradigm shift toward a multimodal, Video-Based Digital Loan Origination and Risk Assessment System.[1] This advanced architecture is engineered to entirely digitize the customer onboarding continuum in real-time. By transitioning the primary user interaction channel from static web forms to a secure, live video communication session, the system leverages continuous audio streams, high-definition video feeds, and geographical metadata to perform deterministic risk evaluations augmented by advanced Large Language Model (LLM) intelligence.[1] The system is designed to seamlessly orchestrate end-to-end loan onboarding, biometric verification, sophisticated risk assessment, and dynamic offer generation with absolute minimal human intervention, effectively democratizing access to credit while fortifying the institution against fraudulent incursions.[1]

This exhaustive specification document maps the complete functional, non-functional, data, and compliance requirements for the Video-Based Digital Loan Origination System. It establishes the architectural blueprint necessary to digitally onboard customers, capture explicit verbal consent, generate precise eligibility signals, and synthesize internal bureau data with AI-driven propensity scores.[1] Furthermore, this document provides a comprehensive,

step-by-step implementation protocol utilizing Google Antigravity—a state-of-the-art agentic Integrated Development Environment (IDE) that shifts the software engineering paradigm from manual syntax generation to the orchestration of autonomous AI agents.[2] By leveraging the provided deployment protocols and master agentic prompts, engineering teams can autonomously scaffold, build, test, and deploy the entire loan origination ecosystem.

Regulatory Landscape and Compliance Architecture

The foundational architecture of the Video-Based Digital Loan Origination System is inextricably linked to the stringent regulatory frameworks governing digital finance. The system must operate in absolute alignment with the Reserve Bank of India (RBI) Master Direction on KYC, specifically adhering to the updated mandates surrounding the Video-based Customer Identification Process (V-CIP).[3] The RBI's continuous updates through 2024 and 2025 have solidified V-CIP as a robust, secure, and inclusive methodology for remote customer identification, account conversion, and periodic KYC updation.[3]

The regulatory environment imposes severe infrastructural and procedural constraints on the system's design. The technology infrastructure facilitating the V-CIP interaction must be strictly housed within the Regulated Entity's (RE) own premises, or, if a cloud deployment model is utilized, the ownership, control, and residency of the data must unequivocally remain with the RE.[3] All video and audio streams must be secured utilizing end-to-end cryptographic encryption to prevent interception or manipulation during transit.[3] Prior to deployment, the entire application perimeter must undergo exhaustive Vulnerability Assessments and Penetration Testing (VAPT), alongside comprehensive security audits conducted by empanelled auditors of the Indian Computer Emergency Response Team (CERT-In).[3]

Operational compliance within the V-CIP framework mandates continuous session integrity and active anti-fraud countermeasures. The system must seamlessly integrate "liveness checks" designed to detect and thwart fraudulent manipulation, such as the use of deepfakes, spoofing algorithms, or pre-recorded video injections.[3] To guarantee that the interaction is occurring in real-time, the system or the authorizing official must dynamically vary the sequence and typology of the questions presented to the prospective borrower.[3] Any visual or auditory evidence of third-party prompting, coaching, or suspicious peripheral conduct at the customer's end must immediately trigger a rejection protocol and terminate the onboarding process.[3] Furthermore, the architecture must handle network volatility gracefully; while momentary pausing of the video feed is permissible provided the underlying cryptographic session remains intact, any complete disconnection or dropped call necessitates the absolute abandonment of the current workflow and the initiation of a completely fresh V-CIP session.[3]

The system must also integrate securely with centralized identification repositories. The validation of the customer's identity relies on specific authorized mechanisms, including Aadhaar One-Time Password (OTP) based e-KYC authentication in a non-face-to-face capacity, offline XML verification of Aadhaar, the retrieval of KYC records from the Central KYC Records Registry (CKYCR) using a unique KYC identifier, or the ingestion of equivalent

e-documents issued directly through DigiLocker.[3] Crucially, to adhere to strict data privacy mandates, the system must automatically implement redaction or masking protocols to ensure that the customer's Aadhaar number is never visible in the raw video footage or stored in plain text within the application databases.[3] If Aadhaar XML files or secure QR codes are utilized, the system must validate that the generation timestamp of these artifacts is not older than three working days from the exact date of the V-CIP interaction.[3]

Core System Architecture and Artificial Intelligence Integration

The Video-Based Digital Loan Origination System operates as a highly concurrent, cloud-native ecosystem comprising specialized microservices. The architecture bridges the synchronous, real-time demands of multimedia streaming with the asynchronous, computationally intensive requirements of machine learning and large language model inference.

The primary interface layer relies on Web Real-Time Communication (WebRTC) protocols to establish low-latency, peer-to-peer or server-relayed media connections between the customer's device and the ingestion servers. This allows the system to capture continuous, high-fidelity video and audio streams directly from the web or mobile browser without requiring external plugin installations. Concurrently, the frontend application continuously polls the device's navigator geolocation API, securely transmitting latitude and longitude coordinates to the backend fraud detection engine.[1]

The processing layer is heavily augmented by specialized artificial intelligence models operating in parallel. The live audio stream is routed through a low-latency Speech-to-Text (STT) transcription engine, which converts the spoken dialogue into a continuous textual stream.[1] This transcript is immediately parsed using deterministic Natural Language Processing (NLP) heuristics to extract critical structured data points, such as the applicant's declared income, employment status, employer name, and the explicit purpose of the requested loan.[1] Simultaneously, the video processing microservice continuously samples individual frames from the live visual feed, routing them into a Computer Vision (CV) inference pipeline.[1] This CV pipeline utilizes deep convolutional neural networks to perform facial analysis, calculating an estimated chronological age based on recognized biological markers, bone structure dimensions, and skin texture.[1] This age estimation acts as an independent, secondary validation signal against the applicant's officially declared age and institutional policy thresholds, immediately flagging significant variances for manual review or automated rejection.[1]

The cognitive core of the system is the Large Language Model (LLM) Intelligence Layer. Once the STT transcript reaches a logical conclusion or a designated milestone in the conversation, the complete text, along with synthesized behavioral metadata (such as pause durations and hesitation markers), is transmitted to the LLM orchestration engine.[1] The LLM performs deep contextual interpretation of the unstructured dialogue, normalizing the applicant's responses and identifying subtle conversational nuances that rigid rules-based engines routinely fail to

detect.[1] The LLM generates a structured JSON intelligence payload that classifies the customer into specific risk bands, constructs a behavioral persona, and assigns a confidence indicator regarding the perceived truthfulness of the interaction.[1] It is architecturally critical that this LLM intelligence layer functions as an augmentative signal; its outputs are synthesized within the final decision engine but possess no authority to override strict, deterministic regulatory rules or hardcoded credit policy exclusions.[1]

Functional Requirements Specification

The functional perimeter of the system is segmented into sequential, interdependent modules that shepherd the applicant from initial entry through to the generation of a personalized financial product. The following subsections exhaustively map the required behaviors and capabilities of the software architecture.

Customer Entry and Telemetry Ingestion

The origination journey must commence with a secure, frictionless entry point designed to maximize marketing campaign conversion rates while immediately establishing cryptographic trust.

Requirement ID	Module	Functional Description	Execution Standard
FR-1.01	Campaign Routing	The system shall generate mathematically unique, cryptographically hashed, and time-bound Uniform Resource Locators (URLs) intended for distribution across multichannel marketing campaigns, including SMS, WhatsApp, targeted emails, and embedded digital interfaces.[1]	SHA-256 Hashing, 24-hour TTL expiration.

FR-1.02	Session Initialization	Upon activation of the secure link, the system shall validate the token integrity, instantiate a secure WebSocket connection for signaling, and initialize the frontend progressive web application.[1]	WebSocket Protocol (WSS), TLS 1.3 encryption.
FR-1.03	Permission Handshake	The application shall explicitly prompt the user for hardware access permissions, specifically targeting the device's camera, primary microphone, and geolocation hardware sensors, failing gracefully with instructional prompts if denied.[1]	HTML5 MediaDevices API, Geolocation API.
FR-1.04	Continuous Telemetry	The system shall silently aggregate and transmit session telemetry throughout the duration of the connection, including IP address origination, browser user-agent strings, precise connection timestamps, and	RESTful POST intervals at 10-second frequencies.

		continuous geolocation polling to detect real-time movement or location spoofing.[1]	

Multimedia Streaming and Biometric Capture

The system's integrity relies entirely on the continuous, uninterrupted flow of high-fidelity multimedia data from the client device to the central processing architecture.

Requirement ID	Module	Functional Description	Execution Standard
FR-2.01	WebRTC Tunneling	The system shall establish an end-to-end encrypted WebRTC media tunnel, negotiating Interactive Connectivity Establishment (ICE) candidates to ensure optimal routing through NATs and firewalls via STUN/TURN servers.[1]	WebRTC, DTLS-SRTP encryption standards.
FR-2.02	Audio Extraction	The media server shall dynamically decouple the audio track from the multiplexed WebRTC stream, downsampling the audio buffer to optimize it for ingestion into the Speech-to-Text	16kHz, Mono channel PCM audio buffering.

		inference engine.[1]	
FR-2.03	Frame Sampling	The video processing daemon shall sample keyframes from the live video stream at predetermined intervals, compressing the images and transmitting them to the Computer Vision API for liveness and age estimation analysis.[1]	1 Frame Per Second (FPS) sampling, WebP compression.
FR-2.04	Liveness Verification	The system shall mandate the user to perform randomized physical actions (e.g., randomized head movements, reading dynamic numerical codes aloud) to continuously verify physical presence and thwart presentation attacks.[3]	Randomized prompt generation via frontend UI overlay.

Conversational Processing and Autonomous Data Entry

To eradicate the friction associated with manual typing, the system autonomously populates the application state using the applicant's spoken words.

Requirement ID	Module	Functional Description	Execution Standard

FR-3.01	Real-Time STT	The system shall maintain an active WebSocket connection to the STT provider, returning intermediate and finalized transcription strings to the frontend for display and to the backend for data extraction.[1]	Sub-500ms transcription latency.
FR-3.02	Entity Extraction	The NLP pipeline shall continuously scan the finalized transcripts to identify and extract Named Entities relevant to the loan application schema, specifically parsing numerical income values, employment sectors, and the stated purpose of the capital.[1]	Custom Named Entity Recognition (NER) models.
FR-3.03	Dynamic Auto-Fill	The frontend architecture shall bind the extracted entities to the React state management layer, visually auto-filling the corresponding fields on the user's screen in real-time to provide immediate visual feedback of the	React state synchronization, optimistic UI updates.

		system's comprehension.[1]	
FR-3.04	Consent Archival	The system shall prompt the user to explicitly state a randomized consent phrase. The STT engine must recognize this phrase with high confidence, triggering the backend to create an immutable log entry that permanently links the consent timestamp to the exact video frame and audio buffer.[1]	SHA-256 cryptographic hashing of the consent artifact.

Multidimensional Risk Assessment and Offer Generation

The culmination of the onboarding process requires the synthesis of all collected data points into a singular, deterministic financial decision.

Requirement ID	Module	Functional Description	Execution Standard
FR-4.01	LLM Contextualization	The system shall construct a comprehensive prompt containing the full interaction transcript, the calculated age estimation variance, and the geo-location risk flags, transmitting	JSON payload structuring, prompt engineering constraints.

		this payload to the LLM orchestration layer for behavioral analysis.[1]	
FR-4.02	Bureau Integration	The risk engine shall execute secure API calls to internal historical databases and external credit bureaus, retrieving the applicant's credit score, historical repayment behavior, and outstanding debt obligations.[1]	Mutual TLS (mTLS) authentication for external APIs.
FR-4.03	Rules Engine Execution	The system shall aggregate the LLM persona classification, the CV age estimate, the extracted income, and the bureau data into a central rules engine. This engine evaluates the payload against hardcoded institutional eligibility parameters and exclusions to calculate a final propensity score.[1]	Deterministic Boolean logic execution trees.
FR-4.04	Dynamic Structuring	Upon successful clearance of the rules engine, the	Standardized financial amortization

		system shall dynamically compute the optimal financial product, generating a personalized JSON object containing the maximum approved loan amount, selectable repayment tenures, the applied annual percentage rate (APR), and the exact Equated Monthly Installment (EMI) schedules.[1]	algorithms.
FR-4.05	Immutable Archival	The final workflow step shall aggregate the raw video file, the audio track, the full text transcript, the generated LLM payload, the bureau response, and the accepted offer into a single encrypted package, storing it within the central audit repository for regulatory review.[1]	AWS S3 / Google Cloud Storage with WORM (Write Once Read Many) policies.

Non-Functional Requirements Specification

The underlying infrastructure must sustain the functional capabilities through rigorous performance, scalability, and security benchmarks.

The architecture must demonstrate elastic scalability, utilizing containerized orchestration (e.g., Kubernetes) to automatically provision additional WebRTC media servers and ML inference pods during peak marketing campaign hours to prevent dropped connections or degraded

video quality.[1] Network latency is a critical constraint; the round-trip time from audio ingestion, through STT processing and LLM classification, back to the frontend UI must reliably remain below 2000 milliseconds to preserve the natural cadence of human conversation during the live call.[1] The system must exhibit profound fault tolerance. In the event of a latency spike or service degradation from the primary LLM provider, the architecture must implement automatic circuit breakers and gracefully degrade to a purely deterministic, rules-based evaluation engine to ensure the onboarding flow is not interrupted.[1]

Security and data privacy form the bedrock of the application. All data traversing the network must be secured utilizing Transport Layer Security (TLS) version 1.3. Data resting within the databases and the immutable audit repository must be encrypted utilizing the Advanced Encryption Standard (AES) with 256-bit keys.[8] Access to the administrative dashboards and the historical audit logs must be strictly governed by Role-Based Access Control (RBAC), ensuring that only authorized compliance officers and specific V-CIP personnel can view sensitive customer telemetry.[1]

Financial Domain Logic and Loan Structuring

The underlying decision engine must accurately reflect the specific financial products offered by Poonawalla Fincorp. The system must process different logic trees depending on whether the applicant is seeking a standard personal loan or a specialized professional loan.

The system must evaluate the applicant against the following structural criteria for Professional Loans:

Evaluation Parameter	Required Threshold	Verification Mechanism
Applicant Age	Between 24 and 65 years	OVD extraction and CV age estimation variance check.[10]
Work Experience	Minimum 1-year post-qualification practice	Extracted from STT transcript and cross-referenced with bureau data.[10]
Minimum Income	₹3 Lakh per annum	Extracted from STT dialogue and verified via digital bank statement ingestion.[10]

During the dynamic offer generation phase (Requirement FR-4.04), the system must utilize the extracted data to calculate precise loan eligibility parameters. The rules engine requires the

applicant's declared profession, the preferred loan tenure (ranging from 12 to 84 months), the net monthly salary, and the sum of all currently existing EMIs.[11] The system calculates the debt-to-income ratio to ensure prudent credit utilization.[11]

Furthermore, the LLM intelligence layer must be programmed to identify conversations indicating a desire for debt consolidation. The system must recognize that personal loans utilized to clear revolving credit card debt are highly advantageous for the consumer. The logic engine should factor in that personal loan interest rates (typically 9.99% to 16% p.a.) are significantly lower than credit card revolving rates (36% to 48% p.a.).[7] If the STT transcript indicates the purpose of the loan is to consolidate credit card debt, the LLM should flag this as a positive propensity indicator, as reducing credit card utilization to zero rapidly repairs the applicant's CIBIL score and signifies responsible financial restructuring.[7] The final offer presented to the user must explicitly display the Annual Percentage Rate (APR) within the Key Fact Statement (KFS), ensuring total transparency regarding processing fees and eliminating hidden leakages.[7]

The Agentic Development Paradigm: Google Antigravity

The implementation, deployment, and continuous iteration of the Video-Based Digital Loan Origination System will be executed utilizing Google Antigravity. Released into public preview in November 2025, Antigravity fundamentally redefines the software engineering workflow, evolving the traditional Integrated Development Environment (IDE) into an autonomous, agent-first platform.[2] Rather than serving merely as a sophisticated autocomplete mechanism, Antigravity empowers developers to act as high-level systems architects, delegating complex, multi-file engineering tasks to autonomous AI agents powered by the Gemini 3.1 Pro and Claude 4.5 Sonnet models.[2]

Architectural Components of Antigravity

The Antigravity platform bifurcates the developer experience into two distinct operational surfaces, designed to support both granular code manipulation and broad system orchestration.[12]

Core Component	Architectural Function	Utility in Project Deployment
The Editor View	A traditional, highly optimized text editor interface (forked from the VS Code foundation)	Utilized for manual oversight, specific bug patching, and the explicit definition of API schemas

	featuring deeply integrated AI Tab completions and inline Command modalities for synchronous, line-by-line code generation.[12]	within the FastAPI backend.
The Agent Manager	A "Mission Control" dashboard facilitating the high-level orchestration of multiple asynchronous AI agents. It allows the developer to spawn, monitor, and direct agents operating across disparate workspaces simultaneously.[12]	Utilized to direct the overarching scaffolding of the Next.js frontend, the PostgreSQL database migrations, and the orchestration of the LLM MCP servers.
Browser Subagent	A seamlessly integrated Chrome browser instance that the AI agent can actuate autonomously. The agent can navigate web pages, click elements, input text, and evaluate Document Object Model (DOM) states to visually verify its own code.[2]	Utilized to autonomously test the WebRTC camera permissions, simulate the video onboarding flow, and verify the dynamic rendering of the loan offer UI.[15]

Execution Modes and Artifact Generation

Antigravity agents operate across two distinct execution paradigms: Fast Mode and Planning Mode. For the construction of complex, full-stack architectures like the Loan Origination System, developers must strictly utilize **Planning Mode**.[14] In this configuration, the agent refrains from immediately writing code. Instead, it conducts deep contextual research across the existing file system, evaluates the user's prompt against structural constraints, and generates a comprehensive implementation plan.[13]

This planning phase introduces the concept of "Artifacts." To build trust and prevent the tedious review of raw terminal logs, Antigravity agents generate tangible, verifiable deliverables—Artifacts—such as architectural markdown documents, specific task lists, visual code diffs, and even recorded WebP videos of the browser subagent executing UI tests.[12] The developer interacts with these Artifacts asynchronously, utilizing Google Docs-style inline

comments to provide precise feedback, correct misunderstandings, or approve the logic before the agent commits the changes to the codebase.[14]

Agent Skills and Global Rules

To prevent context saturation and tool bloat, Antigravity implements a sophisticated system of Workspace Rules and Agent Skills.[18] By defining a GEMINI.md file in the project root, the developer injects a permanent system prompt that dictates the architectural philosophy for the entire project (e.g., enforcing strict type-checking in TypeScript, mandating robust error handling in Python, and prohibiting the hardcoding of API secrets).[19]

Furthermore, the platform utilizes the Model Context Protocol (MCP) to ingest specialized knowledge packages stored within the .agents/skills/ directory.[21] Each skill is encapsulated in a SKILL.md file featuring YAML frontmatter that defines its trigger conditions.[22] This architecture ensures that the agent dynamically loads the precise instructions necessary for configuring the WebRTC tunnels or orchestrating the LLM integration only when those specific tasks are requested, preserving computational bandwidth and significantly reducing cognitive hallucination.[18]

Infrastructure Initialization Protocol

To leverage the agentic capabilities of the platform, the developer must first configure the local environment and install the Antigravity binaries. The platform is distributed across macOS, Windows, and specialized Linux distributions.[2]

Installation Directives

The system administrator must execute the following commands based on the host operating system to install the IDE and the necessary command-line interfaces:

macOS (Apple Silicon / Intel) via Homebrew: The most efficient methodology for macOS environments utilizes the Homebrew package manager to tap the official Google repository and install the application cask.[23]

Bash

```
brew tap google/antigravity
brew install --cask antigravity
```

Linux (Debian/Ubuntu) via APT: Debian-based systems require the manual ingestion of the Google repository signing key to ensure package integrity, followed by the addition of the

source list and execution of the APT installation.[23]

Bash

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://us-central1-apt.pkg.dev/doc/repo-signing-key.gpg | sudo gpg --dearmor
--yes -o /etc/apt/keyrings/antigravity-repo-key.gpg
echo "deb [signed-by=/etc/apt/keyrings/antigravity-repo-key.gpg]
https://us-central1-apt.pkg.dev/projects/antigravity-auto-updater-dev/ antigravity-debian main" | sudo
tee /etc/apt/sources.list.d/antigravity.list > /dev/null
sudo apt update
sudo apt install antigravity
```

Linux (Fedora/RHEL) via RPM: Red Hat Enterprise Linux and Fedora environments necessitate the configuration of the YUM repository parameters prior to utilizing the DNF package manager.[23]

Bash

```
sudo tee /etc/yum.repos.d/antigravity.repo << EOL
[antigravity-rpm]
name=Antigravity RPM Repository
baseurl=https://us-central1-yum.pkg.dev/projects/antigravity-auto-updater-dev/antigravity-rpm
enabled=1
gpgcheck=0
EOL
sudo dnf makecache
sudo dnf install antigravity
```

Upon successful installation, the developer must launch the application, authenticate via a personal Google account to unlock the free preview quota for the Gemini 3.1 Pro and Claude 4.5 models, and initialize an empty directory representing the root of the Loan Origination System.[14] The developer then opens the Agent Manager interface (Cmd + E) to inject the master scaffolding prompt.[25]

Autonomous Project Scaffolding: Master Agentic

Prompt

The capability of Antigravity to generate production-ready code is directly proportional to the structural rigor of the prompt provided.[19] The following prompt is explicitly engineered to instruct the agent to autonomously scaffold the entire Video-Based Digital Loan Origination System.

The developer must ensure that Antigravity is set to **Planning Mode** within the Agent Manager prior to submission.[14] This prompt utilizes XML tags to strictly demarcate identity parameters, contextual constraints, technology stacks, and sequential execution chains.[26]

<identity>

You are Antigravity, a highly advanced agentic AI coding assistant and Principal Full-Stack Solutions Architect operating on the Gemini 3.1 Pro model. You possess exhaustive expertise in modern web frameworks (React, Next.js), asynchronous backend development (Python, FastAPI), real-time multimedia streaming (WebRTC), complex database ORM migrations (SQLAlchemy, Alembic), and the orchestration of Model Context Protocol (MCP) servers for LLM integration. You execute complex software tasks autonomously, prioritizing the creation of robust implementation plans and verifiable Artifacts before writing production code.

</identity>

<context>

You are tasked with building the "Poonawalla Fincorp Loan Wizard," an enterprise-grade Video-Based Digital Loan Origination and Risk Assessment System from scratch. This system replaces legacy, form-based financial onboarding with a live, real-time video call experience. The architecture must continuously capture WebRTC audio and video telemetry, mock the extraction of text from speech (STT), simulate age estimation via computer vision (CV), evaluate applicant risk via an LLM intelligence layer, and dynamically generate instant, personalized loan offers. Absolute adherence to secure coding practices and financial regulatory constraints is mandatory.

</context>

<tech_stack>

Your generated architecture must strictly adhere to the following stack:

1. **Frontend Architecture:** Next.js 16 utilizing the App Router paradigm, React components, strict TypeScript, styling via Tailwind CSS, and accessible component design using Shadcn UI.
2. **Multimedia Integration:** Native browser WebRTC APIs (getUserMedia) for establishing local live video and audio streams.

3. **Backend Architecture:** Python 3.11+ utilizing the FastAPI framework with strict enforcement of asynchronous endpoints (async def).
4. **Data Persistence:** PostgreSQL database schemas modeled via SQLAlchemy ORM, with schema migrations managed exclusively through Alembic.
5. **AI Simulation Layer:** Internal FastAPI routes mocking integration with STT, CV, and LLM services.
6. **Testing Modality:** Automated UI validation utilizing the internal Antigravity Browser Subagent.
   </tech_stack>

<task_chain>

You must execute this architectural build in strict, sequential phases. You are prohibited from progressing to a subsequent phase until the current phase is fully coded, deployed locally, and verified.

CRITICAL INSTRUCTION: Generate a detailed Implementation Plan Artifact outlining all four phases. Wait for my explicit approval via a comment on the Artifact before executing Phase 1.

Phase 1: Environment Initialization and Database Scaffolding

1. Utilize your terminal execution tools to initialize a new Next.js project within the ./frontend directory.
2. Initialize a new Python environment within the ./backend directory, generating a comprehensive requirements.txt containing FastAPI, Uvicorn, SQLAlchemy, Alembic, and Pydantic.
3. Construct the SQLAlchemy database models. You must create the following exact tables with strict typing:
   - CustomerSession (UUID id, secure_token string, status enum, created_at timestamp)
   - TelemetryLog (Foreign Key session_id, ip_address string, latitude float, longitude float, timestamp)
   - TranscriptRecord (Foreign Key session_id, raw_dialogue text, extracted_income integer, extracted_profession string)
   - RiskEvaluation (Foreign Key session_id, cv_age_estimate float, llm_behavioral_persona string, llm_risk_band string, final_approval_decision boolean)
   - LoanOffer (Foreign Key session_id, maximum_amount float, tenure_months integer, interest_rate float, calculated_emi float)
4. Initialize the Alembic environment, generate the initial migration script based on the models, and apply the migration to establish a local SQLite database for development testing.

Phase 2: FastAPI Backend Services and Real-Time Routing

1. Construct the core RESTful API endpoints utilizing FastAPI:
   - POST /api/v1/session/initialize -> Generates a secure, hashed token for the video session.

    - POST /api/v1/telemetry/ingest -> Accepts JSON payloads containing live browser geolocation coordinates and connection metadata.
    - POST /api/v1/ai/process-video-frame -> Accepts Base64 encoded video frames. Mock the response to return a simulated CV age estimation (e.g., 32.5 years) and a confidence score.
    - POST /api/v1/ai/process-audio-chunk -> Accepts audio blobs. Mock the response to return simulated STT transcripts extracting variables like "Income: 5 Lakhs" and "Profession: Doctor".
2. Engineer the LLM Intelligence Endpoint (POST /api/v1/ai/evaluate-risk). This endpoint must ingest the aggregated STT transcript and telemetry logs, constructing a structured prompt for the LLM. Mock the LLM processing to return a strict JSON payload containing the keys risk_band (LOW/MEDIUM/HIGH), persona_classification, and truthfulness_confidence.
3. Construct the POST /api/v1/offer/calculate endpoint. Implement a deterministic rules engine that evaluates the mock AI data: if cv_age_estimate is below 24 or above 65, or if llm_risk_band equals 'HIGH', return a rejection status. Otherwise, utilize standard financial amortization formulas to calculate the EMI based on the extracted income and return the structured Loan Offer.

Phase 3: Next.js Frontend and WebRTC Implementation

1. Develop a high-fidelity frontend landing page reflecting professional, trust-inspiring financial branding.
2. Construct the primary VideoOnboarding React component. You must implement the navigator.mediaDevices.getUserMedia logic to request and bind the user's camera and microphone to an HTML <video> element. Handle permission denials with graceful UI error states.
3. Implement a useEffect hook that captures a still frame from the active video feed every 5000 milliseconds, encodes it to Base64, and POSTs it to the /api/v1/ai/process-video-frame endpoint.
4. Implement a polling mechanism that captures navigator.geolocation coordinates upon component mount and POSTs them to the /api/v1/telemetry/ingest endpoint.
5. Construct a dynamic UI overlay layered over the live video feed. This overlay must display real-time simulated transcriptions, visually auto-fill application fields as data is "extracted," and ultimately render a high-visibility "Personalized Loan Offer" card upon successful backend evaluation.

Phase 4: Autonomous Verification via Browser Subagent

1. Execute the necessary terminal commands to spin up the FastAPI backend (uvicorn main:app --reload) and the Next.js frontend (npm run dev) on separate ports.
2. Deploy the browser_subagent tool. Instruct the subagent to navigate to http://localhost:3000.
3. The subagent must click the "Start Video KYC" initialization button, autonomously accept

the browser media permissions, and verify that the local camera feed is rendered to the DOM.
4. The subagent must verify that the network tab shows successful POST requests to the telemetry and AI processing endpoints, and that the final Loan Offer UI card is visible.
5. Capture a WebP video recording of this subagent interaction and save it to the artifacts directory.
6. Generate a final Markdown Artifact documenting the completed API schemas, database architecture, and a summarized deployment guide.
</task_chain>

<operational_constraints>

- You must strictly utilize absolute file paths relative to the current active workspace when invoking any file creation tools. Never write project files to temporary directories or the .gemini folder.
- You are strictly prohibited from hallucinating external API keys or attempting to execute live LLM inference calls against external providers in the backend codebase; you must utilize robust mocking functions as instructed in Phase 2.
- Ensure all terminal commands are executed sequentially and cleanly. If a command fails, evaluate the stderr output, formulate a correction, and retry autonomously.
</operational_constraints>

Acknowledge these parameters and proceed immediately with the generation of the Phase 1 Implementation Plan Artifact.

By meticulously adhering to the architectural guidelines, regulatory constraints, and agentic deployment protocols detailed within this specification, financial institutions can successfully construct, deploy, and scale a highly secure, frictionless, and AI-driven Video-Based Digital Loan Origination System.

Works cited

1. Agentic AI video Call Based Onboarding.docx
2. Google Antigravity - Wikipedia, accessed April 17, 2026, https://en.wikipedia.org/wiki/Google_Antigravity
3. Know Your Customer (KYC) Direction, 2016 (Updated as on August 14, 2025) - Reserve Bank of India, accessed April 17, 2026, https://www.rbi.org.in/commonman/English/scripts/notification.aspx?id=2607
4. FAQs on Master Direction on KYC - Reserve Bank of India, accessed April 17, 2026, https://www.rbi.org.in/commonman/english/Scripts/FAQs.aspx?Id=3782
5. KYC / AML Policy – 2025-26 - Central Bank of India, accessed April 17, 2026, https://www.centralbankofindia.co.in/sites/default/files/2025-04/FINAL-KYCAML-Policy-2025-26-upto-06.11.2024.pdf
6. AI Age Estimation in KYC: Accuracy, Privacy & Compliance | Ondato, accessed

April 17, 2026, https://ondato.com/blog/age-estimation-in-kyc/
7. Personal Loans India 2026: Interest Rates & Eligibility Guide - Stashfin, accessed April 17, 2026, https://www.stashfin.com/blogs/how-to-get-personal-loan-india
8. How to Write an Effective Software Requirements Specification - Dataforest, accessed April 17, 2026, https://dataforest.ai/blog/software-requirements-specification-understandable-framework
9. IEEE Standard for Software Requirements Specifications (IEEE 830–1998) | by Abdul Rehman | Medium, accessed April 17, 2026, https://medium.com/@abdul.rehman_84899/ieee-standard-for-software-requirements-specifications-ieee-830-1998-0395f1da639a
10. Professional Loan Eligibility Criteria & Required Documents - Poonawalla Fincorp, accessed April 17, 2026, https://poonawallafincorp.com/professional-loan/eligibility-and-documents
11. Personal Loan Eligibility Calculator Online – Check Eligibility - Poonawalla Fincorp, accessed April 17, 2026, https://poonawallafincorp.com/personal-loan/eligibility-calculator
12. Build with Google Antigravity, our new agentic development platform, accessed April 17, 2026, https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/
13. Tutorial : Getting Started with Google Antigravity | by Romin Irani - Medium, accessed April 17, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-google-antigravity-b5cc74c103c2
14. Getting Started with Google Antigravity, accessed April 17, 2026, https://codelabs.developers.google.com/getting-started-google-antigravity
15. Google Antigravity Prompts - GitHub Gist, accessed April 17, 2026, https://gist.github.com/CypherpunkSamurai/f16e384ed1629cc0dd11fea33e444c17
16. How to Set Up and Use Google Antigravity - Codecademy, accessed April 17, 2026, https://www.codecademy.com/article/how-to-set-up-and-use-google-antigravity
17. Introducing Google Antigravity, a New Era in AI-Assisted Software Development, accessed April 17, 2026, https://antigravity.google/blog/introducing-google-antigravity
18. Authoring Google Antigravity Skills, accessed April 17, 2026, https://codelabs.developers.google.com/getting-started-with-antigravity-skills
19. GEMINI.md & System Prompt Guide: Rules That Work (2026) - Antigravity Codes, accessed April 17, 2026, https://antigravity.codes/blog/antigravity-gemini-md-system-prompt-guide
20. Rules / Workflows - Google Antigravity Documentation, accessed April 17, 2026, https://antigravity.google/docs/rules-workflows
21. rmyndharis/antigravity-skills: A curated collection of Agent ... - GitHub, accessed

April 17, 2026, https://github.com/rmyndharis/antigravity-skills
22. Agent Skills - Google Antigravity Documentation, accessed April 17, 2026, https://antigravity.google/docs/skills
23. Download Antigravity v1.20.5: Mac, Windows & Linux (2026), accessed April 17, 2026, https://antigravity.codes/download
24. Google Antigravity Download Linux, accessed April 17, 2026, https://antigravity.google/download/linux
25. Google Antigravity Documentation, accessed April 17, 2026, https://antigravity.google/docs
26. system-prompts-and-models-of-ai-tools/Google/Antigravity/Fast Prompt.txt at main - GitHub, accessed April 17, 2026, https://github.com/x1xhlol/system-prompts-and-models-of-ai-tools/blob/main/Google/Antigravity/Fast%20Prompt.txt